SAHARA: Real-Time On-Device Sound Event Detection and Alerting for Deaf and Hard-of-Hearing Users, with India-Specific Acoustic Adaptation

Mahi Mayani, Advika Mishra, Prachi Teotia, Raj Singh, Priti Rumao
Department of Computer Engineering
Mukesh Patel School of Technology Management & Engineering (MPSTME)
SVKM's NMIMS (Deemed-to-be University)
Mumbai, India
priti.rumao@nmims.edu

Abstract—Deaf and hard-of-hearing (DHH) people miss safety-critical sounds such as smoke alarms and sirens, yet sound-recognition aids rarely report how often they alert falsely and are seldom tested outside Western homes. We present SAHARA, a fully on-device Android system that runs a frozen YAMNet encoder with a 296k-parameter classifier head for eight household sound classes and sends class-coded vibration alerts to the phone and to a proposed low-cost ESP32 wristband. Each class's threshold and temporal rule are tuned jointly against an explicit budget of false alarms per hour, with safety classes weighted three times. On held-out public data the system reaches 0.778 mAP at 3.5 false alarms per hour of continuous background audio, and the tuning cuts false alarms from 34.1 to 6.1 per hour on the same model. We also contribute SAHARA-IN, 329 sound clips and 107 name-call clips recorded in Mumbai homes and streets. Cross-validated mAP on them rises from 0.621 to 0.822 when recordings from the same sessions join training, but not when whole sessions are held out (0.600); a three-recording home calibration step instead raises the share of events alerted in unseen homes from 19 % to 35 %. A speech-embedding spotter tells users when their name is called and by whom.

Index Terms—assistive technology, deaf and hard of hearing, sound event detection, on-device machine learning, haptic feedback

I. Introduction

More than 1.5 billion people live with some degree of hearing loss and over 430 million with disabling loss [1]; in India an estimated 63 million people have significant auditory impairment [2]. For them, everyday sounds that carry safety or social information (a smoke alarm, a siren, a doorbell, a baby crying, someone calling their name) are inaudible unless converted to a visual or tactile signal. DHH users consistently rank alarms and safety sounds as the most important to be notified about and prefer glanceable, wearable and private notifications [3]–[5].

Deep audio embeddings trained on AudioSet [6] have made sound recognition on phones and watches practical [7]–[10]. Three gaps remain. First, systems are evaluated with symmetric, threshold-free metrics, yet what users experience is a stream of alerts: a missed fire alarm can be fatal, and frequent false alerts lead users to ignore the aid [7], [11]. Second, many prototypes send audio or features to a server, which is unacceptable for an always-listening device in the home. Third, public corpora come mostly from Western environments; Indian homes add pressure-cooker whistles, temple bells, hawkers and monsoon rain, and their doorbells are often melodic chimes.

SAHARA addresses these gaps with five contributions: (1) alert rules tuned against an explicit false-alarm budget; (2) fully on-device inference that never stores audio; (3) name-call alerts for any voice, naming enrolled callers; (4) SAHARA-IN, a field set from Indian homes, evaluated with a session-held-out protocol, and a home-calibration step; and (5) an open wristband design whose protocol and firmware are implemented and whose hardware is our next step.

II. Related Work

Sound awareness for DHH users. Matthews et al. studied visualisations of non-speech sound [12], and Bragg et al. designed a personalisable mobile sound detector [3]. In a survey of 201 DHH participants, Findlater et al. found strong interest in sound awareness on wearable and mobile devices [4], and Goodman et al. evaluated smartwatch sound feedback across contexts [5]. Jain et al. deployed HomeSound in homes and found that false alerts quickly erode trust [7]; SoundWatch moved recognition to a smartwatch and analysed accuracy, latency and battery trade-offs [8]. Huang et al. ran a three-week field study of mobile sound recognition with DHH participants and reported that background noise, sound variability and confusable classes remain obstacles [11]. ProtoSound personalises recognition from a few examples with prototypical networks [9], [13], Goodman et al. studied user-driven personalisation [14], and UbiEar brought location-independent sound awareness to smartphones [15]. Earlier wearable prototypes also used smartwatches as the delivery device [16]. These systems report accuracy on clips rather than alerts per hour of real audio, and were evaluated in North American or European homes.

Audio embeddings and detection. VGGish [17], OpenL3 [18], PANNs [19], the Audio Spectrogram Transformer

[20] and YAMNet [21], a MobileNetV1 [22] trained on AudioSet, provide transferable embeddings; data augmentation and weak labels are standard tools for small environmental-sound sets [23], [24]. Ubicoustics showed that such models transfer to real-world activity recognition [10]. We use YAMNet because it runs continuously on Android phones and exports to TensorFlow Lite [25]. FSD50K [26], AudioSet [6], ESC-50 [27] and DESED [28] are standard sources of domestic events; Mesaros et al. review detection metrics [29], [30]. Cost-sensitive learning is well established [31]; we carry the cost asymmetry past the loss into the alert rule itself.

Keyword spotting and speaker recognition. Speech embeddings pretrained for keyword spotting let a small head learn a new word from few examples [32], as packaged by openWakeWord [33]; GE2E training gives compact speaker embeddings for verification [34].

III. System Design

Fig. 1 shows the pipeline. A native Kotlin foreground service captures 16 kHz mono audio, so detection continues with the screen off. Samples enter a 1 s RAM ring buffer that advances every 0.5 s; a mean-square energy gate (10^{-4}) skips silent hops. Each remaining 0.975 s patch is embedded by a frozen YAMNet into a 1024-dimensional vector, and the same pass yields YAMNet's 521 AudioSet scores. The mean of the two latest non-silent embeddings feeds a head (Dense 256 and 128 with ReLU and dropout, Dense 8 with sigmoid) of 296,328 parameters that scores smoke alarm, doorbell, siren, knocking, dog bark, baby cry, glass break and appliance beep independently. The training code computes exactly the same window vectors as the phone (tested to a maximum difference of 0.0), and 54 test clips replayed through the app's code raise the same alerts as the evaluation pipeline in 54 of 54 cases.

Privacy. Audio exists only in the ring buffer and is overwritten every 0.5 s; nothing is written to storage, the release build has no Internet permission, and enrolment keeps only embedding vectors.

Name calls and open-set notices. When YAMNet's speech score is at least 0.2, pretrained speech embeddings [32], [33] feed a logistic head that detects the user's name ("Aryan"). A GE2E speaker encoder [34] then compares the voice with family members enrolled once on the phone: at cosine similarity 0.70 or more the alert reads "Raj is calling you", otherwise "Someone is calling you". Sounds outside the eight classes that YAMNet recognises with score ≥ 0.35 (e.g., a tap running) produce a quiet notice without vibration, so they never compete with safety alerts.

Haptics. Each alert reaches the wristband as four bytes (version, command, class, intensity), and the phone and wristband play the same class-coded pattern (Fig. 2): the smoke alarm uses the "temporal-three" rhythm of evacuation alarms [35], a siren swells, a doorbell is a strong-then-soft "ding-dong" and knocking is four taps. The wristband (ESP32, coin motor driven through a MOSFET, 500 mAh cell, protected charger; about Rs. 870) is designed and its firmware compiles; it has not yet been built or measured.

IV. Safety-Weighted Alerting

Smoke alarm, siren and glass break are safety-critical: their class weight is $w_c = 3$, baby cry and appliance beep 1.5, the rest 1. The head is trained with a positive-weighted binary cross-entropy $\mathcal{L} = -\frac{1}{C}\sum_c [w_c y_c \log \hat{y}_c + (1 - y_c)\log(1 - \hat{y}_c)]$, so a missed positive costs up to three times a false positive [31]. Sample weights balance classes and up-weight background windows eight times and SAHARA-IN clips ten times. Adam (learning rate 3×10^{-4}, batch 128) runs with early stopping.

Alert rule. Class c fires when its current window scores $s_{c,t} \geq \theta_c$, at least $k_c \in \{1, 2, 3\}$ of its last three windows are hits, and it has not fired within 10 s. With a 1 s window, 0.5 s hop and two pooled embeddings, the worst-case decision latency for a sustained event is $W + 2H + (k_c - 1)H + \tau = 2.0$–$3.0\,\text{s} + \tau$, where τ is the inference time.

Tuning against a budget. Instead of a fixed threshold we replay every validation clip through the exact rule and, for each candidate (θ_c, k_c), measure the detection rate D_c (share of the class's clips that raise its alert) and the false-alarm rate A_c (alerts per hour on validation audio without the class). We then solve

$$\max_{\{\theta_c, k_c\}} \sum_c w_c\, D_c(\theta_c, k_c) \quad \text{s.t.} \quad \sum_c A_c(\theta_c, k_c) \leq B \qquad (1)$$

with a Lagrangian sweep and choose $B = 4$ per hour (Fig. 3). Run selection also uses only validation data: the best tuned score among runs trained on the current data, with safety-class precision of at least 0.70. Solving the same problem at $B \in \{1, 2, 4, 8, 16\}$ gives the app's five-step sensitivity setting, so every step has a measured trade-off.

V. Data and Protocol

Public data. Target clips come from FSD50K, AudioSet and ESC-50; background comes from 10,000 FSD50K clips without target-like labels, the 44 non-target ESC-50 categories and DESED soundscapes (Table I). We verified every label identifier against the official vocabularies, which removed serious errors in an earlier build (e.g., AudioSet "glass" identifiers pointing to other classes and FSD50K's generic Alarm mapped to appliance beep). Every class has at least 150 real clips and 20 in test. Splits are 70/15/15, grouped by upstream recording (YouTube or Freesound identifier) so one recording never appears in two splits. Each training target clip is also mixed onto three training-split backgrounds at -5 to 10 dB SNR (6,834 mixtures); a separate set of 564 test mixtures is reported apart from real data.

Processing. All audio is resampled to 16 kHz mono. Each clip is cut into the same 1 s windows the phone sees,

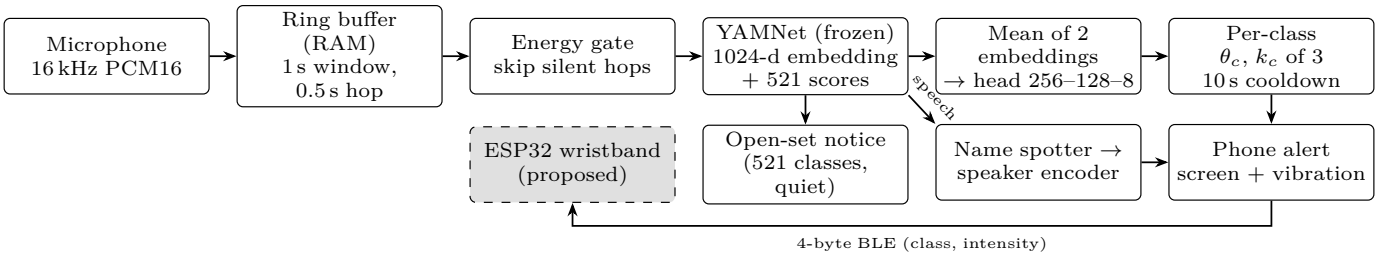


Fig. 1. SAHARA pipeline. Everything runs on the phone in an Android foreground service; only a class identifier and an intensity byte cross the Bluetooth link to the proposed wristband (dashed).

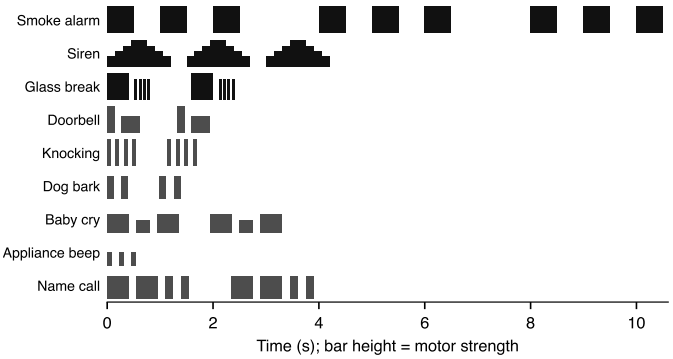


Fig. 2. Vibration patterns on one time scale, from the firmware; bar height is motor strength. Safety classes (top) use full strength.

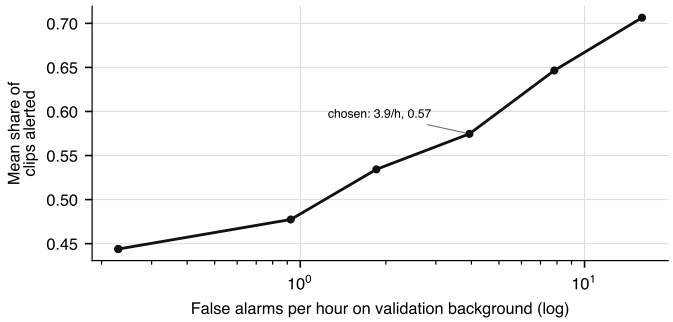


Fig. 3. Mean share of validation clips alerted against the false-alarm budget (log scale), same model.

embedded once by YAMNet and cached (22,521 clips). For weakly labelled clips, only the windows where YAMNet's own score for the class is highest are kept as positives; the rest of a target clip is dropped rather than used as background.

SAHARA-IN. Four team members recorded 329 clips on their own phones in Mumbai homes and streets, in three uploads: 269 target clips (67 knocking, 59 appliance beep, 58 doorbell, 20 each of smoke alarm, siren and glass break, 14 baby cry, 11 dog bark) and 60 background clips (hawkers, a temple, a kitchen, television, monsoon rain, drawers and doors). About 40 % of each label is held out for test, and each upload is split on its own so new clips never move earlier ones. A further 107 clips (38 of the name "Aryan", 69 of similar names) serve the name-call model, evaluated leave-one-recorder-out. Because the recordings come from few sittings, we infer each clip's recording session (its recorder, or for unattributed uploads the phone's numbered file-name series) and also report cross-validation that holds out whole sessions.

TABLE I
Real clips per class (test clips in brackets); † safety-critical. Background also includes 605 DESED clips.

Class	AudioSet	ESC-50	FSD50K	SAHARA-IN	Total (test)
Smoke alarm†	183	–	–	20	203 (35)
Doorbell	92	–	144	58	294 (57)
Siren†	34	40	132	20	226 (42)
Knocking	97	40	373	67	577 (100)
Dog bark	10	40	536	11	597 (94)
Baby cry	106	40	–	14	160 (30)
Glass break†	97	40	510	20	667 (105)
Appliance beep	261	40	206	59	566 (101)
Background	–	1,760	10,000	60	12,425 (1,876)

Metrics. We report per-class AP over 0.5 s windows, its mean (mAP), precision, recall and F1 at the tuned thresholds, the share of test clips that raise the correct alert under the deployed rule, and false alarms per hour with background audio played as one continuous stream. The test split was used once.

VI. Results

Detection. On the public test set the model reaches 0.778 mAP and a macro F1 of 0.670 (Table II), and played as a stream through the deployed rule it raises 3.5 false alarms per hour on 3.4 h of test background, mostly "siren". The safety classes are strongest: 22 of 27 smoke-alarm and 28 of 34 siren clips raise an alert. Knocking (AP 0.604) and appliance beep (0.567) are weakest; their brief, varied sounds limit the model rather than the rule (Fig. 3). On the synthetic mixtures at −5 to 10 dB, mAP falls to 0.666, so loud background remains the main challenge. Max-pooling instead of averaging the two embeddings, tried to keep brief sounds from being diluted, changed nothing (validation score 0.609 vs. 0.614 over five seeds).

What the tuning buys. Replaying all 2,435 real test clips (4.6 h) through three rules on the same model (Table III; false alarms here count alerts for a class on any audio without it, so they exceed the background-only figure),

TABLE II
Public test set per class at the deployed rule: threshold θ_c, windows k_c of 3, window AP and F1, and clips alerted.

Class	θ_c	k_c	AP	F1	Clips alerted
Smoke alarm†	0.81	2	0.916	0.858	22/27
Doorbell	0.995	1	0.711	0.432	13/35
Siren†	0.91	1	0.875	0.817	28/34
Knocking	0.95	1	0.604	0.410	21/73
Dog bark	0.98	1	0.908	0.847	67/90
Baby cry	0.99	2	0.817	0.712	17/25
Glass break†	0.93	1	0.827	0.790	56/97
Appliance beep	0.90	3	0.567	0.496	17/78
Mean			0.778	0.670	0.56

TABLE III
Same model, three alert rules, all real test audio.

Alert rule	Clips	Safety	FA/h
$\theta = 0.5$, 1 window	0.766	0.838	49.1
$\theta = 0.5$, 1 (safety) or 2 of 3 windows	0.688	0.838	34.1
Tuned (θ_c, k_c), $B = 4$/h (SAHARA)	0.538	0.692	6.1

a fixed threshold of 0.5 with 2-of-3 smoothing gives 34.1 false alarms per hour, more than one every two minutes. Budget tuning brings this to 6.1 while still alerting on 54 % of clips and 69 % of safety clips. The sensitivity steps span 47 % of clips at 2.5 false alarms per hour to 67 % at 20.6.

Indian conditions. In five-fold cross-validation over the 329 SAHARA-IN clips with random folds, adding them to training raises mAP from 0.621 to 0.822 and the clips alerted from 45 to 100 of 269; smoke alarm AP rises from 0.385 to 0.801 and doorbell from 0.521 to 0.824. Holding out whole recording sessions instead, the gain disappears (0.621 vs. 0.600), although doorbells still improve (0.521 to 0.691). The recordings therefore teach the model a particular home's sounds far better than Indian sounds in general, and the held-out test split (0.831 mAP; 0.698 for a model trained before the latest upload) overstates transfer to a new home.

Home calibration. This motivates a calibration step: the user records a sound three times where it happens; the app sets that class's threshold 0.5 log-odds below the weakest recording's firing level, never below its threshold at the 16-per-hour sensitivity step, and an optional recording of ordinary home sound raises it again if needed. On session-held-out models (three clips of an unseen session as calibration, the rest as events, 20 draws), the share of events alerted across 17 session–class pairs rises from 19 % to 35 %, for about one extra false alarm per hour per calibrated class. Doorbells gain most (e.g., 20 % to 83 %); knocking and glass break do not, because their scores stay below even the most permissive tested threshold.

Name calls. Leaving one recorder out, the name spotter separates calls of "Aryan" from background scenes with ROC AUC 0.872 (equal error rate 19 %) and from similar names such as Arya and Aryanshi with AUC 0.743 (32 %). Caller identification names the right person in 29 of 38 calls; with a caller not enrolled, 82 % of their calls read "Someone".

On-device cost. On a Samsung Galaxy S24 (Android 16), inference takes 20–29 ms per window with the screen on and 47–49 ms with it off, the models occupy 15.0 MB (YAMNet), 1.2 MB (head) and 8.1 MB (name call), the release APK is 45.8 MB, and memory use is 128.9 MB. In a speaker test the phone alerted 1.0–2.5 s after sound onset, within the latency bound above.

VII. Discussion and Limitations

At 4 false alarms per hour the model alerts on about 58 % of validation events, and quadrupling the budget raises this only to 71 %, so brief sounds need a stronger model (e.g., fine-tuning YAMNet's last layers) or more data, not a looser rule. SAHARA-IN is small, with 20 clips per safety class, and comes from few sessions, so claims about unseen Indian homes need many more households. The name spotter is trained for one name and confuses similar names about a third of the time; its speech models are licensed for non-commercial use. We have not yet built the wristband, measured battery drain over a day, or run a study with DHH participants; a field deployment in Mumbai measuring missed events, alert fatigue and recognition of the haptic patterns is planned [4], [11].

VIII. Conclusion

SAHARA shows that sound awareness for DHH users can run entirely on a phone, retain no audio and be tuned for the number that decides whether users keep it on: false alarms per hour. Tuning each class's threshold and temporal rule against an explicit budget cut false alarms more than five-fold on the same model. Field recordings from Indian homes taught the model those homes' sounds but not yet unseen ones, and a three-recording calibration step recovers much of that gap. A separate speech path tells users when they are called and by whom. Code, alert tuning, Bluetooth protocol, firmware and the SAHARA-IN protocol are released.

References

[1] World Health Organization, World Report on Hearing. Geneva: WHO, 2021.

[2] Directorate General of Health Services, Ministry of Health & Family Welfare, Government of India, "National Programme for Prevention and Control of Deafness (NPPCD)." [Online]. Available: https://dghs.mohfw.gov.in

[3] D. Bragg, N. Huynh, and R. E. Ladner, "A personalizable mobile sound detector app design for deaf and hard-of-hearing users," in Proc. ACM SIGACCESS Conf. Computers and Accessibility (ASSETS), 2016, pp. 3–13.

[4] L. Findlater, B. Chinh, D. Jain, J. Froehlich, R. Kushalnagar, and A. C. Lin, "Deaf and hard-of-hearing individuals' preferences for wearable and mobile sound awareness technologies," in Proc. CHI Conf. Human Factors in Computing Systems, 2019.

[5] S. M. Goodman, P. Kirchner, R. Guttman, D. Jain, J. Froehlich, and L. Findlater, "Evaluating smartwatch-based sound feedback for deaf and hard-of-hearing users across contexts," in Proc. CHI Conf. Human Factors in Computing Systems, 2020.

[6] J. F. Gemmeke et al., "Audio Set: An ontology and human-labeled dataset for audio events," in Proc. IEEE ICASSP, 2017, pp. 776–780.

[7] D. Jain et al., "HomeSound: An iterative field deployment of an in-home sound awareness system for deaf or hard of hearing users," in Proc. CHI Conf. Human Factors in Computing Systems, 2020.

[8] D. Jain, H. Ngo, P. Patel, S. Goodman, L. Findlater, and J. Froehlich, "SoundWatch: Exploring smartwatch-based deep learning approaches to support sound awareness for deaf and hard of hearing users," in Proc. ACM ASSETS, 2020.

[9] D. Jain et al., "ProtoSound: A personalized and scalable sound recognition system for deaf and hard-of-hearing users," in Proc. CHI Conf. Human Factors in Computing Systems, 2022.

[10] G. Laput, K. Ahuja, M. Goel, and C. Harrison, "Ubicoustics: Plug-and-play acoustic activity recognition," in Proc. ACM UIST, 2018.

[11] J. Z. Huang, H. Chhabria, and D. Jain, "'Not there yet': Feasibility and challenges of mobile sound recognition to support deaf and hard-of-hearing people," in Proc. ACM ASSETS, 2023, pp. 1–14.

[12] T. Matthews, J. Fong, F. W.-L. Ho-Ching, and J. Mankoff, "Evaluating non-speech sound visualizations for the deaf," Behav. Inf. Technol., vol. 25, no. 4, pp. 333–351, 2006.

[13] J. Snell, K. Swersky, and R. Zemel, "Prototypical networks for few-shot learning," in Proc. NeurIPS, vol. 30, 2017.

[14] S. M. Goodman et al., "Toward user-driven sound recognizer personalization with people who are d/Deaf or hard of hearing," Proc. ACM Interact. Mob. Wearable Ubiquitous Technol., vol. 5, no. 2, 2021.

[15] S. Liu, Z. Zhou, J. Du, L. Shangguan, J. Han, and X. Wang, "UbiEar: Bringing location-independent sound awareness to the hard-of-hearing people with smartphones," Proc. ACM Interact. Mob. Wearable Ubiquitous Technol., vol. 1, no. 2, 2017.

[16] M. Mielke and R. Brück, "A pilot study about the smartwatch as assistive device for deaf people," in Proc. ACM ASSETS, 2015, pp. 301–302.

[17] S. Hershey et al., "CNN architectures for large-scale audio classification," in Proc. IEEE ICASSP, 2017, pp. 131–135.

[18] J. Cramer, H.-H. Wu, J. Salamon, and J. P. Bello, "Look, listen, and learn more: Design choices for deep audio embeddings," in Proc. IEEE ICASSP, 2019, pp. 3852–3856.

[19] Q. Kong, Y. Cao, T. Iqbal, Y. Wang, W. Wang, and M. D. Plumbley, "PANNs: Large-scale pretrained audio neural networks for audio pattern recognition," IEEE/ACM Trans. Audio, Speech, Lang. Process., vol. 28, pp. 2880–2894, 2020.

[20] Y. Gong, Y.-A. Chung, and J. Glass, "AST: Audio spectrogram transformer," in Proc. Interspeech, 2021, pp. 571–575.

[21] M. Plakal and D. Ellis, "YAMNet," TensorFlow Model Garden, 2020. [Online]. Available: https://github.com/tensorflow/models/tree/master/research/audioset/yamnet

[22] A. G. Howard et al., "MobileNets: Efficient convolutional neural networks for mobile vision applications," arXiv:1704.04861, 2017.

[23] J. Salamon and J. P. Bello, "Deep convolutional neural networks and data augmentation for environmental sound classification," IEEE Signal Process. Lett., vol. 24, no. 3, pp. 279–283, 2017.

[24] A. Kumar and B. Raj, "Audio event detection using weakly labeled data," in Proc. ACM Multimedia, 2016, pp. 1038–1047.

[25] B. Jacob et al., "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in Proc. IEEE CVPR, 2018, pp. 2704–2713.

[26] E. Fonseca, X. Favory, J. Pons, F. Font, and X. Serra, "FSD50K: An open dataset of human-labeled sound events," IEEE/ACM Trans. Audio, Speech, Lang. Process., vol. 30, pp. 829–852, 2022.

[27] K. J. Piczak, "ESC: Dataset for environmental sound classification," in Proc. ACM Multimedia, 2015, pp. 1015–1018.

[28] N. Turpault, R. Serizel, A. Parag Shah, and J. Salamon, "Sound event detection in domestic environments with weakly labeled data and soundscape synthesis," in Proc. Detection and Classification of Acoustic Scenes and Events Workshop (DCASE), 2019, pp. 253–257.

[29] A. Mesaros, T. Heittola, and T. Virtanen, "Metrics for polyphonic sound event detection," Applied Sciences, vol. 6, no. 6, p. 162, 2016.

[30] A. Mesaros, T. Heittola, T. Virtanen, and M. D. Plumbley, "Sound event detection: A tutorial," IEEE Signal Process. Mag., vol. 38, no. 5, pp. 67–83, 2021.

[31] C. Elkan, "The foundations of cost-sensitive learning," in Proc. IJCAI, 2001, pp. 973–978.

[32] J. Lin, K. Kilgour, D. Roblek, and M. Sharifi, "Training keyword spotters with limited and synthesized speech data," in Proc. IEEE ICASSP, 2020, pp. 7474–7478.

[33] D. Scripka, "openWakeWord," GitHub, 2023. [Online]. Available: https://github.com/dscripka/openWakeWord

[34] L. Wan, Q. Wang, A. Papir, and I. Lopez Moreno, "Generalized end-to-end loss for speaker verification," in Proc. IEEE ICASSP, 2018, pp. 4879–4883.

[35] ISO 8201:2017, Alarm systems—Audible emergency evacuation signal—Requirements. Geneva: ISO, 2017.